

DDA5001 Machine Learning

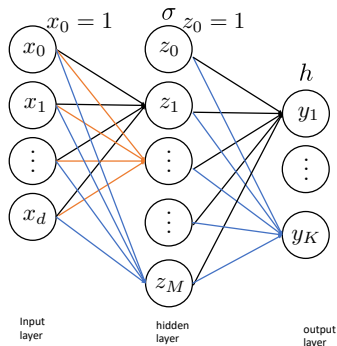
Neural Networks (Part II): Training Formulation and BP

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Recap: One Hidden Layer (Two-Layer) Neural Network



Use $\mathbf{V}$ and $\mathbf{W}$ to denote the weight matrices of the first layer and second layer. NN model can be written as

$$\mathbf{y} = f_{\theta}(\mathbf{x}) = h(\mathbf{W}\sigma(\mathbf{V}\mathbf{x})).$$

- The input to the next layer is the output of the previous layer ($\mathbf{z} = \sigma(\mathbf{V}\mathbf{x})$).

Recap: Ingredients and Interpretation of Neural Network

Activation function:

- ▶ σ and h are activation functions, but have quite different functioning.
- ▶ Typical choices for σ are sigmoid, tanh, ReLU, etc.
- ▶ The choice of h depends on applications, which is to impose either linear regression or linear classification in the last layer using the learned feature z .

Interpretation: One can think of neural network model as extracting features by nonlinear network and **finally put the extracted feature z into the last layer for linear regression or linear classification.**

Universal approximation power: One hidden layer (two layer) NN can approximate almost arbitrary target g .

Agenda

Training Two Layer Neural Networks

Backpropagation

Training Neural Networks

- ▶ Training data: $(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_n, \mathbf{y}_n)$ with $\mathbf{x}_i \in \mathbb{R}^d$ and $\mathbf{y}_i \in \mathbb{R}^K$
- ▶ Neural network model

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = h(\mathbf{W}\sigma(\mathbf{V}\mathbf{x})).$$

- ▶ We aim to learn the weight parameters $\boldsymbol{\theta} = (\mathbf{V}, \mathbf{W})$ such that

$$\mathbf{y}_i \leftarrow f_{\boldsymbol{\theta}}(\mathbf{x}_i).$$

This is a supervised learning problem.

- ▶ We can quantify this approximation by choosing a **loss function** which we will seek to minimize by picking $\boldsymbol{\theta}$ appropriately

The Learning Problem for Training Neural Networks

Regression: $h(t) = t$ and use squared error (ℓ_2) loss, resulting in

► $K = 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\boldsymbol{\theta}}(\mathbf{x}_i))^2 \right\}.$$

The Learning Problem for Training Neural Networks

Regression: $h(t) = t$ and use squared error (ℓ_2) loss, resulting in

► $K = 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\boldsymbol{\theta}}(\mathbf{x}_i))^2 \right\}.$$

► $K > 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - f_{\boldsymbol{\theta}}(\mathbf{x}_i)\|_2^2 \right\}.$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function.
MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \log(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) \right\}.$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function.
MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \log(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) \right\}.$$

Classification (Multi-class): $K > 1, \mathbf{y} = (0, \dots, 1, \dots, 0)$, h is soft-max.
MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \mathbf{y}_i^{\top} \log(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) \right\}.$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function.
MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \log(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) \right\}.$$

Classification (Multi-class): $K > 1, \mathbf{y} = (0, \dots, 1, \dots, 0)$, h is soft-max.
MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^n \mathbf{y}_i^{\top} \log(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) \right\}.$$

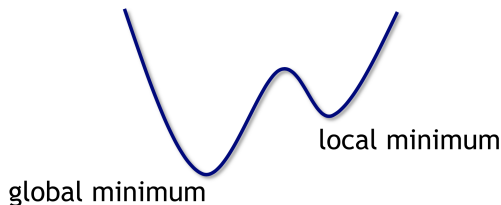
Summary: NN learning formulation is the same as least squares or logistic regression, with the difference being using $\mathbf{z} = \sigma(\mathbf{V}\mathbf{x})$ as input.

Training Neural Networks is Nonconvex Optimization

For example, regression training problem can be written as:

$$\min_{\theta=(\mathbf{V}, \mathbf{W})} \left\{ \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - h(\mathbf{W}\sigma(\mathbf{V}\mathbf{x}_i))\|^2 \right\}.$$

This is a highly **nonconvex optimization** problem.



Recall that linear supervised learning always gives rise to convex optimization. The nonconvexity of NN learning comes from **learning the feature $\mathbf{z} = \sigma(\mathbf{V}\mathbf{x})$** , where $\mathbf{V}$ is part of the learnable parameters.

Training Neural Networks

We put an abstract form of the former learning problems:

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta}) \right\}$$

For different applications (regression or classification), ℓ_i has its own form.

Training Neural Networks

We put an abstract form of the former learning problems:

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta}) \right\}$$

For different applications (regression or classification), ℓ_i has its own form.

- ▶ One can apply a gradient-based training algorithm.
- ▶ One needs to compute the gradient

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\boldsymbol{\theta}).$$

- ▶ Apply gradient-based training algorithm:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k).$$

However... The gradient is not easy to compute and GD training algorithm might be too optimistic.

Next: Training Algorithm and its Ingredients

We next dive into the depth of neural network training:

- ▶ How to compute the gradient?
 $\rightsquigarrow$ The well-known **backpropagation (BP)**.
- ▶ In contemporary applications, n is so large. Applying GD is not feasible.
 $\rightsquigarrow$ **Stochastic gradient descent, Adagrad, and Adam family.**

Agenda

Training Two Layer Neural Networks

Backpropagation

Computing The Gradient: Squared ℓ_2 -Loss as An Example

- ▶ Let us take the regression case where we use squared error (ℓ_2) loss as an example. The conclusion applies to other cases.
- ▶ We consider the general case where $K > 1$. We have

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{y}_i - f_{\boldsymbol{\theta}}(\mathbf{x}_i)\|^2 = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta})$$

where

$$\ell_i(\boldsymbol{\theta}) = \|\mathbf{y}_i - f_{\boldsymbol{\theta}}(\mathbf{x}_i)\|_2^2$$

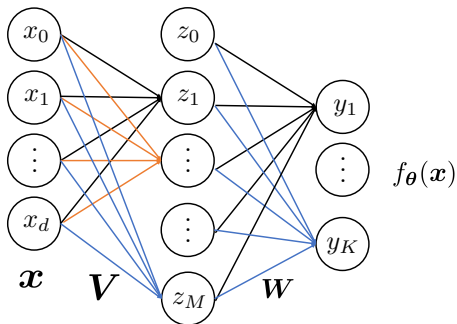
- ▶ Knowing how to take gradient for each ℓ_i suffices.

For ease of notation, we will omit the subscript i in ℓ_i and denote

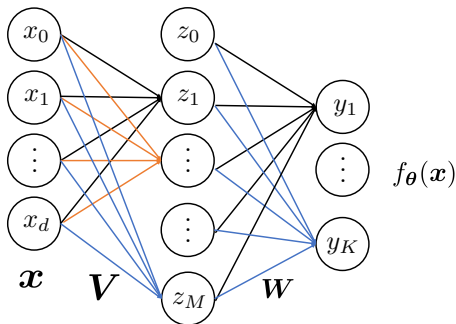
$$\ell(\boldsymbol{\theta}) = \|\mathbf{y} - f_{\boldsymbol{\theta}}(\mathbf{x})\|^2.$$

Task: Computing $\nabla \ell(\boldsymbol{\theta}) = (\frac{\partial \ell}{\partial \mathbf{V}}, \frac{\partial \ell}{\partial \mathbf{W}})$.

Backpropagation for Gradient Computation: Overview



Backpropagation for Gradient Computation: Overview



The idea: Computing gradient using **chain rule**:

- **Forward pass:** Given an $\mathbf{x}$, compute (using the current parameters $\mathbf{V}$ and $\mathbf{W}$) $\mathbf{z} = \sigma(\mathbf{V}\mathbf{x})$, $f_{\theta}(\mathbf{x}) = h(\mathbf{W}\mathbf{z})$, $\ell(\theta) = \|\mathbf{y} - f_{\theta}(\mathbf{x})\|_2^2$
- **Backward pass:**
 - Second layer: compute $\frac{\partial \ell}{\partial \mathbf{W}}$, and compute $\frac{\partial \ell}{\partial \mathbf{z}}$.
 - First layer: Compute $\frac{\partial \ell}{\partial \mathbf{V}} = \frac{\partial \ell}{\partial \mathbf{z}} \times \frac{\partial \mathbf{z}}{\partial \mathbf{V}}$.

Deriving The Gradient: Second Layer

- ▶ We omit the offset w.l.o.g. i.e., no x_0 and z_0 .
- ▶ By definition of gradient, we have

$$\frac{\partial \ell}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial \ell}{\partial W(1,1)} & \cdots & \frac{\partial \ell}{\partial W(1,M)} \\ \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial W(K,1)} & \cdots & \frac{\partial \ell}{\partial W(K,M)} \end{bmatrix} \in \mathbb{R}^{K \times M}.$$

- ▶ We focus on one element $\frac{\partial \ell}{\partial W(k,m)}$ of $\frac{\partial \ell}{\partial \mathbf{W}}$ for simplicity.
- ▶ Note that during the forward pass, we have computed

$$\ell(\boldsymbol{\theta}) = \|\mathbf{y} - h(\mathbf{W}\mathbf{z})\|_2^2,$$

where $\mathbf{z} = \sigma(\mathbf{V}\mathbf{x}) \in \mathbb{R}^M$.

Deriving The Gradient: Second Layer

$$\begin{aligned}\frac{\partial \ell}{\partial W(k, m)} &= \frac{\partial}{\partial W(k, m)} \|h(\mathbf{W}\mathbf{z}) - \mathbf{y}\|_2^2 \\&= \frac{\partial}{\partial W(k, m)} \sum_{j=1}^K (h(\mathbf{w}_j^\top \mathbf{z}) - y[j])^2 \\&= \frac{\partial}{\partial W(k, m)} (h(\mathbf{w}_k^\top \mathbf{z}) - y[k])^2 \quad \text{using chain rule} \rightarrow \\&= \frac{\partial (h(\mathbf{w}_k^\top \mathbf{z}) - y[k])^2}{\partial (h(\mathbf{w}_k^\top \mathbf{z}) - y[k])} \times \frac{\partial (h(\mathbf{w}_k^\top \mathbf{z}) - y[k])}{\partial \mathbf{w}_k^\top \mathbf{z}} \times \frac{\partial \mathbf{w}_k^\top \mathbf{z}}{\partial W(k, m)} \\&= 2 (h(\mathbf{w}_k^\top \mathbf{z}) - y[k]) \times h'(\mathbf{w}_k^\top \mathbf{z}) \times z[m] \\&:= \delta[k] \times z[m],\end{aligned}$$

where $\mathbf{w}_k^\top \in \mathbb{R}^M$ is the k -th row of $\mathbf{W}$ and we have defined

$$\delta[k] = 2 (h(\mathbf{w}_k^\top \mathbf{z}) - y[k]) h'(\mathbf{w}_k^\top \mathbf{z}).$$

Deriving The Gradient: Second Layer

Since

$$\frac{\partial \ell}{\partial W(k, m)} = \delta[k] z[m]$$

Given $\delta[k] = 2 (h(\mathbf{w}_k^\top \mathbf{z}) - y[k]) h'(\mathbf{w}_k^\top \mathbf{z})$, denote

$$\boldsymbol{\delta} = \begin{bmatrix} \delta[1] \\ \delta[2] \\ \vdots \\ \delta[K] \end{bmatrix} = 2(h(\mathbf{W}\mathbf{z}) - \mathbf{y}) \odot h'(\mathbf{W}\mathbf{z}) \in \mathbb{R}^K$$

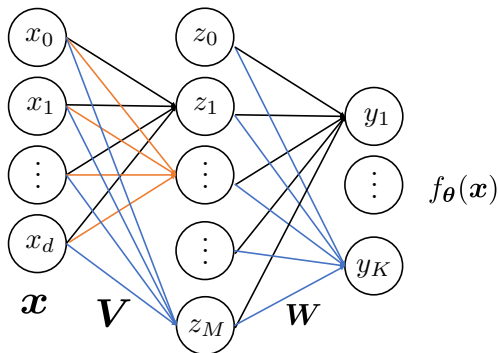
where $\odot$ is the Hadamard (element-wise) product.

We have

$$\frac{\partial \ell}{\partial \mathbf{W}} = \underbrace{\boldsymbol{\delta}}_{K \times 1} \underbrace{\mathbf{z}^\top}_{1 \times M} \in \mathbb{R}^{K \times M},$$

which can be calculated using the current values of $\mathbf{W}$ and $\mathbf{z}$.

Deriving The Gradient: First Layer



$$\ell(\theta) = \|\mathbf{y} - f_{\theta}(\mathbf{x})\|_2^2, f_{\theta}(\mathbf{x}) = h(\mathbf{W}\mathbf{z}), \mathbf{z} = \sigma(\mathbf{V}\mathbf{x})$$

We firstly back-propagate the gradient back to $\mathbf{z}$, and then calculate the gradient of $\mathbf{z}$ w.r.t. $\mathbf{V}$

$$\frac{\partial \ell}{\partial \mathbf{V}} = \frac{\partial \ell}{\partial \mathbf{z}} \times \frac{\partial \mathbf{z}}{\partial \mathbf{V}}$$

Deriving The Gradient: First Layer

To begin with, we back-propagate the gradient back to $\mathbf{z}$.

For an individual node of $\mathbf{z}$:

$$\begin{aligned}\frac{\partial \ell}{\partial \mathbf{z}[m]} &= \frac{\partial}{\partial \mathbf{z}[m]} \|h(\mathbf{W}\mathbf{z}) - \mathbf{y}\|_2^2 \\ &= \frac{\partial}{\partial \mathbf{z}[m]} \sum_{k=1}^K (h(\mathbf{w}_k^\top \mathbf{z}) - y[k])^2 \quad \text{using chain rule} \rightarrow \\ &= \sum_{k=1}^K 2 (h(\mathbf{w}_k^\top \mathbf{z}) - y[k]) \times h'(\mathbf{w}_k^\top \mathbf{z}) \times w_k[m] \\ &= \sum_{k=1}^K \delta[k] w_k[m]\end{aligned}$$

Combining the individual nodes of $\mathbf{z}$, we denote

$$\frac{\partial \ell}{\partial \mathbf{z}} = \mathbf{W}^\top \boldsymbol{\delta}$$

Deriving The Gradient: First Layer

$$\frac{\partial \ell}{\partial \mathbf{V}} = \begin{bmatrix} \frac{\partial \ell}{\partial V(1,1)} & \cdots & \frac{\partial \ell}{\partial V(1,d)} \\ \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial V(M,1)} & \cdots & \frac{\partial \ell}{\partial V(M,d)} \end{bmatrix} \in \mathbb{R}^{M \times d}.$$

For an individual element of $\mathbf{V}$:

$$\begin{aligned} \frac{\partial \ell}{\partial V(m,j)} &= \underbrace{\frac{\partial \ell}{\partial z[m]}}_{\text{gradient from next layer}} \times \underbrace{\frac{\partial z[m]}{\partial V(m,j)}}_{\text{local gradient}} \quad (\text{chain rule}) \\ &= \frac{\partial \ell}{\partial z[m]} \times \frac{\partial \sigma(\mathbf{v}_m^\top \mathbf{x})}{\partial V(m,j)} \quad (\text{since } \mathbf{z} = \sigma(\mathbf{V}\mathbf{x})) \\ &= \sum_{k=1}^K \delta[k] w_k[m] \times \sigma'(\mathbf{v}_m^\top \mathbf{x}) \times x[j] \end{aligned}$$

Combining the gradients of the individual elements:

$$\frac{\partial \ell}{\partial \mathbf{V}} = \underbrace{\left((\mathbf{W}^\top \boldsymbol{\delta}) \odot \sigma'(\mathbf{V}\mathbf{x}) \right)}_{M \times 1} \times \underbrace{\mathbf{x}^\top}_{1 \times d} \in \mathbb{R}^{M \times d}$$

The Overall Procedure

- ▶ Recall that for each data $\mathbf{x}_i$, we need to compute its corresponding gradient $\nabla \ell_i(\boldsymbol{\theta})$.
- ▶ Since $\boldsymbol{\theta} = (\mathbf{V}, \mathbf{W})$, we have

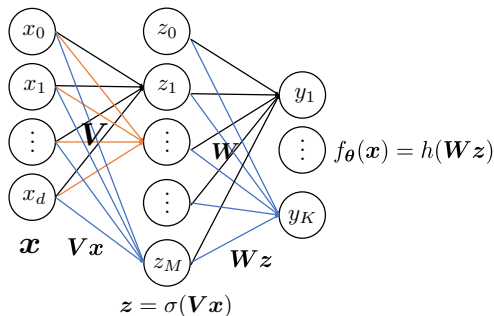
$$\nabla \ell_i(\boldsymbol{\theta}) = \left(\frac{\partial \ell_i}{\partial \mathbf{V}}, \frac{\partial \ell_i}{\partial \mathbf{W}} \right)$$

- ▶ This is achieved by an efficient **two-pass** method for computing the gradient at each iteration—the so-called **backpropagation**:
 1. **Forward pass**
 2. **Backward pass**

Backpropagation: Forward pass

Forward pass: Feed x_i into the network, use the current parameters V and W to compute and store

- ▶ Vx_i
- ▶ $z_i = \sigma(Vx_i)$
- ▶ Wz_i
- ▶ $f_{\theta}(x_i) = h(Wz_i), \ell_i(\theta) = \|y - f_{\theta}(x_i)\|_2^2$



Backpropagation: Backward pass

Backward pass:

Compute the gradient of the second layer

$$\blacktriangleright \delta_i = 2(h(\mathbf{W}\mathbf{z}_i) - \mathbf{y}_i) \odot h'(\mathbf{W}\mathbf{z}_i)$$

$$\blacktriangleright \frac{\partial \ell_i}{\partial \mathbf{W}} = \delta_i \mathbf{z}_i^\top$$

Backpropagate the gradient to $\mathbf{z}_i$:

$$\blacktriangleright \frac{\partial \ell}{\partial \mathbf{z}_i} = \mathbf{W}^\top \delta_i$$

Compute the gradient of the first layer:

$$\blacktriangleright \frac{\partial \ell_i}{\partial \mathbf{V}} = \left(\frac{\partial \ell}{\partial \mathbf{z}_i} \odot \sigma'(\mathbf{V}\mathbf{x}_i) \right) \mathbf{x}_i^\top$$

Thus, **backpropagation** is an efficient way of computing the gradient of NN learning problem.

Given the computed gradient, we can apply gradient-based training algorithm for training NN. $\leadsto$ Which algorithm should we choose?

Structured Learning Problem: Finite-Sum Structure

In LS, LR, SVM, and **NN**, we always have

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \underbrace{\ell(\boldsymbol{\theta}; \mathbf{x}_i, y_i)}_{\ell_i(\boldsymbol{\theta})}$$

each $\ell_i(\boldsymbol{\theta})$ is called a **component function**. We can use **BP** to compute $\nabla \mathcal{L}(\boldsymbol{\theta})$ if it is NN.

Structured Learning Problem: Finite-Sum Structure

In LS, LR, SVM, and **NN**, we always have

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \underbrace{\ell(\boldsymbol{\theta}; \mathbf{x}_i, y_i)}_{\ell_i(\boldsymbol{\theta})}$$

each $\ell_i(\boldsymbol{\theta})$ is called a **component function**. We can use **BP** to compute $\nabla \mathcal{L}(\boldsymbol{\theta})$ if it is NN.

Example: Least squares

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 = \frac{1}{n} \sum_{i=1}^n \underbrace{(\boldsymbol{\theta}^\top \mathbf{x}_i - y_i)^2}_{\ell_i(\boldsymbol{\theta})}$$

- ▶ Each ℓ_i corresponds to one training data point $(\mathbf{x}_i, y_i)$.
- ▶ The above structure is called **finite-sum** and the problem is called **finite-sum optimization**.
- ▶ Finite-sum optimization is ubiquitous in machine learning, including most of the (supervised) learning problems.

Modern Large-Scale Machine Learning Problems

One of the key features of modern machine learning problems is **large-scale**, i.e., **the number of training data n is very large** (guided by VC analysis). Such a problem is called **large-scale machine learning**.

- ▶ One notable example is **training NN**.

We have the following two observations:

- ▶ The property we want for an algorithm is fast convergence. In terms of optimization, we should use fast algorithm (**fast in terms of iteration**) like first-order method (GD/AGD) or even second-order method (Newton).
- ▶ Guided by VC analysis, we just need to choose a fine $\mathcal{H}$ with small generalization error and then design a fast learning algorithm (GD/AGD or Newton) to learn $\hat{\theta}$ by solving the learning problem. Then, the learning process is done.

What is missed?

Modern Large-Scale Machine Learning Problems

One of the key features of modern machine learning problems is **large-scale**, i.e., **the number of training data n is very large** (guided by VC analysis). Such a problem is called **large-scale machine learning**.

- ▶ One notable example is **training NN**.

We have the following two observations:

- ▶ The property we want for an algorithm is fast convergence. In terms of optimization, we should use fast algorithm (**fast in terms of iteration**) like first-order method (GD/AGD) or even second-order method (Newton).
- ▶ Guided by VC analysis, we just need to choose a fine $\mathcal{H}$ with small generalization error and then design a fast learning algorithm (GD/AGD or Newton) to learn $\hat{\theta}$ by solving the learning problem. Then, the learning process is done.

What is missed? We do not consider **the computational issue** of the learning algorithm **when n is large-scale**.

Let us take GD as an example.

Gradient Descent in Large-scale Finite-sum Structure

GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- What is the structure of $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$?

Gradient Descent in Large-scale Finite-sum Structure

GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- What is the structure of $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$?

Due to the **linearity** of gradient operator, we have

$$\begin{aligned}\nabla \mathcal{L}(\boldsymbol{\theta}_k) &= \nabla \left(\frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta}_k) \right) \\ &= \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\boldsymbol{\theta}_k)\end{aligned}$$

- **Observation:** The gradient of $\mathcal{L}$ is the summation of the gradients of **all** the component functions.
 - It can be **costly and even prohibitive** to compute the **full gradient** when **n is very large**.
 - Newton's method is even worse when n is so large.
- ↪ Motivates the design of stochastic optimization methods, next lecture.